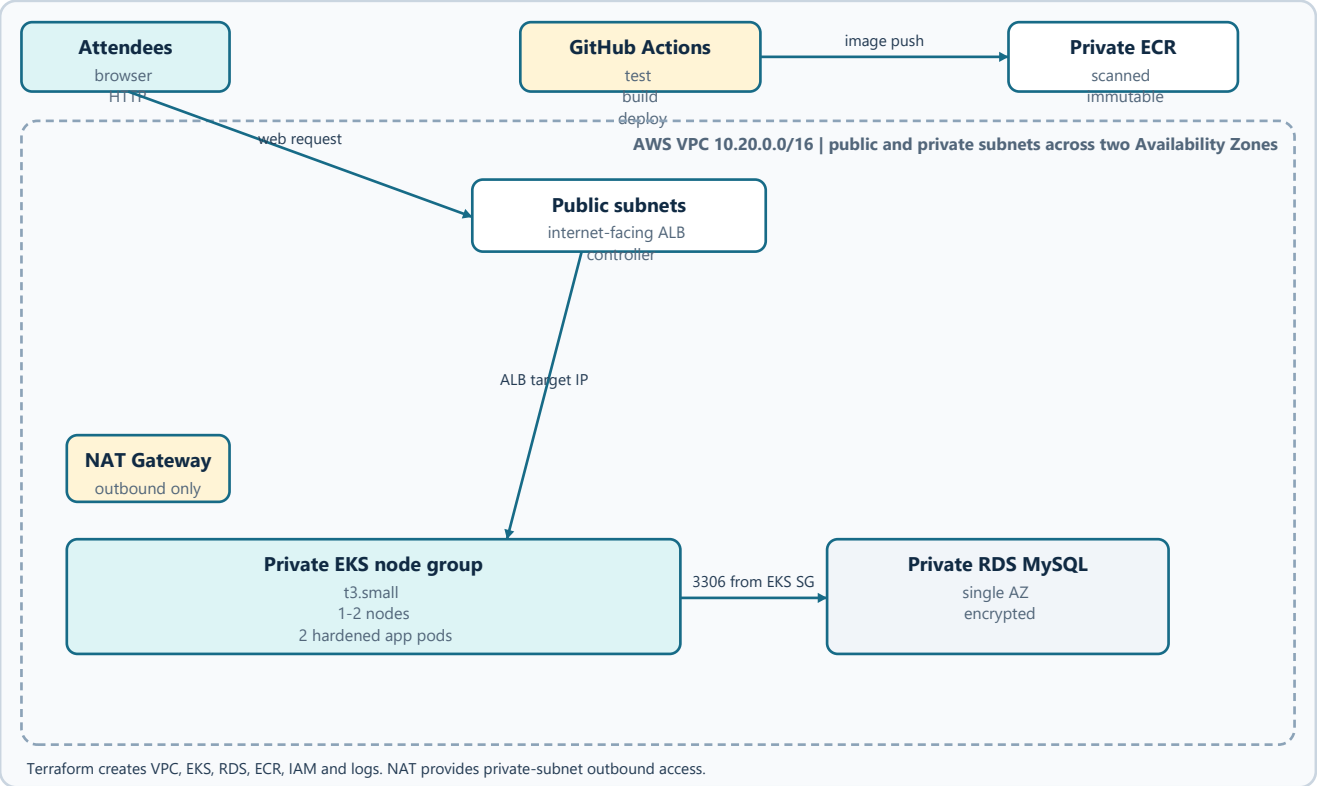
EVENTPASS / EKS CAPSTONE

Architecture and Deployment Guide

Terraform foundation, application delivery, CI/CD flow and security boundaries

A teaching map for launching an online event ticketing demo on AWS EKS. The application supports event browsing, no-payment registrations and an organizer dashboard.



At a glance: Terraform creates the AWS foundation. GitHub Actions tests changes and, after a successful main-branch or manual run, builds an immutable container and deploys it to EKS. A public ALB routes visitors to private Kubernetes pods; MySQL stays in private subnets.

This guide follows the checked-in Terraform and workflow configuration. The environment is sized for short classroom demonstrations, not production workloads or real attendee data.

1. What Terraform creates

Terraform provisions the AWS foundation. Kubernetes manifests and the AWS Load Balancer Controller complete the application-facing layer after the cluster is ready. Terraform state is the inventory of managed resources and must be retained across apply and destroy operations.

Layer	Created or configured	Purpose
Network	VPC 10.20.0.0/16; two AZs; two public and two private subnets; one NAT Gateway; DNS	Public ALB entry and private placement for nodes and database; NAT provides outbound access.
Kubernetes	EKS 1.36; managed group in private subnets; desired/minimum 1, max 2; t3.small	AWS manages control plane; EC2 worker nodes run Kubernetes workloads.
Database	RDS MySQL db.t3.micro; 20 GB gp3; encrypted; private; backup retention 0; no final snapshot	Registration storage. RDS manages the master password in Secrets Manager.
Registry	Private ECR; AES-256 encryption; scan on push; immutable image tags	Stores versioned images built by CI.
Identity and logs	GitHub OIDC provider and deploy role; EKS access entry; CloudWatch control-plane logs, 7 days	Short-lived CI identity, limited deploy authorization and diagnostics.

What is visible to learners

• A temporary public ALB URL where attendees can view events and register without paying.

• An organizer login and dashboard showing registration counts.

• Two app replicas in namespace eventpass, a ClusterIP Service and an Ingress reconciled by the AWS Load Balancer Controller.

Scope: controller/IRSA setup, namespace and runtime Kubernetes Secrets use documented bootstrap/deployment steps. These are not all resources created by the Terraform root module.

2. Network request path and application runtime

The attendee path is short. The AWS Load Balancer Controller watches the Kubernetes Ingress and creates an internet-facing Application Load Balancer. The ALB targets pod IPs, forwarding traffic through Kubernetes networking to the application.

• Browser to ALB: visitor opens the temporary ALB DNS name. The current classroom listener is HTTP; do not use it for sensitive real-world traffic.

• ALB to EKS: the ALB health-checks /healthz and forwards requests to healthy pod IP targets.

• Service to pods: the ClusterIP Service gives the app a stable in-cluster address. The Deployment maintains two replicas.

• Pods to MySQL: app reads non-secret DB settings from a ConfigMap and DB password from a Kubernetes Secret. RDS security group permits port 3306 only from the EKS node security group.

• Outbound traffic: private nodes use NAT for required internet egress. NAT does not provide inbound access to private resources.

Pod security and reliability

Area	Configured behavior
Identity	UID/GID 10001; no privilege escalation; drop all Linux capabilities.
Filesystem	Read-only root filesystem; writable /tmp volume.
Isolation	RuntimeDefault seccomp; service-account token automount disabled.
Reliability	Startup/readiness/liveness probes; CPU and memory requests/limits; two replicas.

EKS API endpoint: public and private endpoint access are enabled. The current config does not restrict the public endpoint by source CIDR. IAM authentication and EKS authorization still apply. For production, narrow the CIDR to instructor/admin IPs or use private-only connectivity.

3. CI/CD: a code change to running pods

Pull requests run tests only. A push to main or manual workflow dispatch can run deployment after tests pass, provided required GitHub repository variables exist. Terraform provisioning is a separate operator workflow.

1	Change and review PR starts a MySQL 8.4 service, installs Python dependencies and runs pytest.
2	Merge or dispatch Successful tests gate deploy. Merge to main or request workflow_dispatch.
3	Federated AWS identity GitHub OIDC token is exchanged by AWS STS for short-lived credentials by assuming eventpass-github-deployer. No long-lived AWS access keys are stored in GitHub.
4	Build and publish Runner builds the container, signs in to ECR and pushes tag commit-SHA + run-ID + attempt. Unique tag supports ECR immutability.
5	Configure and apply Runner updates kubeconfig, applies the DB connection ConfigMap, sets the image URI/tag in Kustomize and runs kubectl apply -k.
6	Wait and inspect Workflow waits up to five minutes for Deployment rollout and prints Ingress. ALB DNS is the temporary attendee URL.

GitHub repository variables

Variable	Set from
AWS_REGION; EKS_CLUSTER_NAME; ECR_REPOSITORY	Terraform outputs / lab config
AWS_DEPLOY_ROLE_ARN	Terraform output github_deployer_role_arn
DB_HOST; DB_NAME; DB_USER	Terraform output RDS endpoint, database name and username

Refresh DB_HOST after recreating RDS because the endpoint may change. These values are configuration, not secrets. The DB password and app secrets are never read by the GitHub deploy job.

4. Security boundaries and tradeoffs

Boundary	Protection in this lab	Limit / next step
GitHub to AWS	OIDC; exact audience and repo/environment trust conditions; short-lived role.	Scoped ECR upload and EKS cluster description permissions; no permanent access keys.
GitHub to Kubernetes	EKS access entry maps role to eventpass-deployer; namespace RBAC grants deployment resources.	Not cluster-admin. Namespace permissions do not include Secret reads.
Database	Private RDS; encrypted storage; SG allows 3306 only from EKS nodes; password in Secrets Manager.	Single AZ; no backup retention; destroy deletes data. Never use real attendee data.
Kubernetes secrets	Instructor bootstrap creates eventpass-db and eventpass-app. GitHub does not read values.	K8s Secret data is base64, not encryption by itself. Enable KMS envelope encryption and tight access in production.
Container supply chain	ECR encryption, push scanning, immutable tags; app runs non-root, read-only and with resource limits.	Review scan results; pin third-party Actions to commit SHAs for stricter supply-chain control.
Public entry	Internet-facing ALB for classroom access.	HTTP only. Add TLS with ACM and managed DNS before any real public use.
EKS API	Public and private endpoints, IAM authentication and EKS access policy.	Public CIDRs are unrestricted in this lab. Restrict or make private-only for production.
Operations	7-day control-plane logs; one NAT; small worker; separate Terraform lifecycle.	These are cost tradeoffs, not HA. Production needs multi-AZ DB, backups, monitoring, budgets and alerts.

Security summary: public traffic enters at the ALB, while application compute and MySQL remain private. CI identity is federated and short-lived. App secrets are provisioned separately by the instructor. Add TLS, narrow EKS API access, backups/Multi-AZ, secret encryption/rotation, monitoring and stricter action pinning before production use.

5. Repeatable classroom launch and cleanup

Use the repository README and scripts as the command source of truth. Review each Terraform plan before applying. The admin EC2 can manage the cluster; students should not use permanent AWS administrator credentials.

Launch sequence

• Clone the repo and enter its root. Confirm AWS identity and region.

• In terraform/, prepare variables and backend per README. Run terraform init, validate and plan; review resource names and spend.

• Apply the foundation. Save cluster, region, ECR, deploy role, RDS endpoint/name/user and network outputs. Preserve state and lock file.

• Configure kubectl and wait for nodes/add-ons to become Ready. Install AWS Load Balancer Controller with IRSA; bootstrap namespace and runtime Secrets using the repo script.

• Set GitHub repository variables from current outputs. Do not add AWS keys or DB passwords to GitHub. Confirm classroom environment and workflow permissions.

• Run workflow_dispatch or merge tested code to main. Verify pods, Service, Ingress and ALB. Test registration and organizer dashboard.

Verification on the admin host

```
aws sts get-caller-identity
aws eks update-kubeconfig --name eventpass --region ap-south-1
kubectl get nodes -o wide
kubectl get pods,svc,ingress -n eventpass
kubectl rollout status deployment/eventpass -n eventpass
```

Cleanup and cost control

• Remove Kubernetes Ingress/app and controller-managed resources as documented before Terraform destroy, so ALBs and target groups are cleaned up.

• Review Terraform state and destroy plan from terraform/. RDS is configured without a final snapshot; destroying it permanently deletes its data.

• Check for remaining ALBs, NAT gateways, EBS volumes, ECR images and CloudWatch log groups. EKS, NAT, ALB and RDS incur charges while running.

• If rebuilding, refresh GitHub DB_HOST from new Terraform output and confirm nodes/add-ons are Ready before CI deploy.

Repository and official references

Repo: terraform/*.tf | .github/workflows/pipeline.yml | k8s/ | scripts/bootstrap-runtime-secrets.sh | README.md | docs/
EKS networking | Managed node groups | EKS access entries
RDS MySQL | AWS Load Balancer Controller | GitHub OIDC to AWS